Build FestPass on your laptop, then let Jenkins build it

A hands-on Java lab for third-year computer science students. The app decides whether a college-fest booking can be accepted. Students make a Git commit, watch eight tests run in Jenkins, and download a runnable JAR. Then they introduce and fix a boundary bug to see red and green builds.

Two ZIPs: extract FestPass_Java_Project.zip and Jenkins_Laptop_Setup.zip into the same parent folder. Neither ZIP includes third-party executables, credentials, an existing Git history, or a configured Jenkins home.

Jenkins laptop lab: commit -> test -> build -> JAR

This folder contains **setup scripts and instructions**, not the Jenkins binary, plugins, a JDK, passwords, or an already configured Jenkins home. Download the software from the official links below. First-time setup needs internet access; after installation, the example can run entirely on one laptop.

The matching project is `festpass-java`. Use only your own trusted teaching code. This is a local learning environment, not a production Jenkins setup.

1. Prerequisites and official downloads

Item	Requirement	Official download / reference
Java	Full **JDK 21**, 64-bit; not just a JRE. JDK 25 is also supported by this pinned Jenkins release.	https://adoptium.net/temurin/releases/?version=21
Jenkins	LTS WAR file. This kit pins **2.580.1**, the LTS release checked on **October 25, 2026**.	https://www.jenkins.io/download/
Exact Jenkins WAR	Our download script downloads this official file and checks its SHA-256.	https://get.jenkins.io/war-stable/2.580.1/jenkins.war
Jenkins Java policy	Check this before substituting other Java or Jenkins releases. Java 17 is not sufficient for the pinned release.	https://www.jenkins.io/doc/book/platform-information/support-policy-java/
Git	Already installed; ensure command-line Git includes `git daemon`.	https://git-scm.com/downloads
IDE	Already installed; use any Java-capable IDE or a text editor. A C++ toolchain is not used.	No new IDE required.
Terminal	Windows PowerShell 5.1+; or Bash + curl on macOS/Linux. macOS uses `shasum`; Linux usually has `sha256sum`.	Usually supplied by the OS / Git installation.
Browser	Any recent desktop browser.	No browser extension required.
Jenkins plugins	Git (`git`), Pipeline (`workflow-aggregator`), JUnit (`junit`) and their dependencies.	https://plugins.jenkins.io/

No Maven, Gradle, Docker, Node.js, Python, database, GitHub account, public repository, cloud subscription, or credit card is needed.

Suggested lab allowance (not a vendor minimum): an 8 GB RAM laptop, around 2 GB free disk, and 30-60+ minutes for first-time downloads/configuration. The guided red/green exercise takes about 15-20 minutes once setup is ready. These scripts cap the controller heap at 768 MB and agent heap at 512 MB; the IDE, compiler, OS, and browser need additional memory.

The required local ports are **8080** (Jenkins HTTP) and **9418** (read-only Git daemon). Both servers bind to **127.0.0.1 only**. Do not expose them through port forwarding, tunnels, public Wi-Fi interfaces, or firewall rules. The inbound

agent uses WebSocket over 8080; port 50000 is not required.

2. Extract the two ZIPs together

Extract the project ZIP and setup ZIP into the **same parent directory**. Avoid nesting one folder inside the other.

Windows example:

```
C:\CloudJenkinsLab\
  festpass-java\
    Jenkinsfile
    ci\Build.java
    src\...
  jenkins-lab\
    Download-Jenkins.ps1
    Start-Jenkins.ps1
    Start-GitServer.ps1
    Start-Agent.ps1
    ...
```

macOS/Linux: use a directory such as `$HOME/CloudJenkinsLab` with the same two child folder names. ZIP extraction may remove executable permissions; the instructions explicitly call `bash`, so `chmod` is not required.

3. Check Java and run the app first

Install Temurin **JDK 21** for your OS and CPU (x64 versus ARM64). On Windows, enable the installer options for PATH and JAVA_HOME, then open a new terminal. If you installed a ZIP build, set JAVA_HOME to the JDK directory and put its `bin` directory on your user PATH. Do not point JAVA_HOME at `bin`.

In every terminal that will start Jenkins or the agent, verify:

```
java -version
javac -version
jar --version
git --version
```

`java` and `javac` should both report 21.x (or both 25.x if deliberately using that alternative). An IDE can use a different bundled JDK; the **terminal versions** determine this lab's behavior.

Open a terminal in `festpass-java`:

```
java ci/Build.java verify
java -jar build/festpass.jar 99 100
java -jar build/festpass.jar 100 100
git init -b main
git add .
git commit -m "Add FestPass app and pipeline"
```

Expected: **8 tests, 0 failures**, a JAR, acceptance for 99/100, and rejection for 100/100. If Git needs an identity, set your own name/email in this repository. Keep generated output out of Git; `.gitignore` is supplied.

4. Terminal A: serve the committed Git repository

Windows PowerShell, from `jenkins-lab`:

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\Start-GitServer.ps1
```

macOS/Linux, from `jenkins-lab`:

```
bash start-git-server.sh
```

The execution-policy flag applies only to that PowerShell process. If school policy blocks local scripts, ask the instructor/administrator rather than changing machine-wide security settings.

Keep this terminal open. In another terminal, check:

```
git ls-remote git://127.0.0.1:9418/festpass-java
```

You should see a commit hash and `refs/heads/main`. This service publishes only this explicitly exported repository and permits read/clone/fetch, not push. It does not send code to an external service. No credentials are needed.

Do not use a local filesystem URL in the Jenkins job or disable Jenkins' local-checkout security protection. The loopback Git protocol avoids that workaround. If the folder name or port changes, use the URL printed by the launcher everywhere below.

5. Terminal B: download and start Jenkins

Windows PowerShell, from `jenkins-lab`:

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\Download-Jenkins.ps1
powershell -NoProfile -ExecutionPolicy Bypass -File .\Start-Jenkins.ps1
```

macOS/Linux:

```
bash download-jenkins.sh
bash start-jenkins.sh
```

Keep the terminal open. Visit **http://127.0.0.1:8080/**. The setup password is printed during startup and is stored at:

```
jenkins-lab/runtime/jenkins-home/secrets/initialAdminPassword
```

Unlock Jenkins, choose **Install suggested plugins**, and create your own administrator account. Keep authentication enabled. Set the Jenkins URL to `http://127.0.0.1:8080/`. Do not share your password or upload Jenkins home.

Under **Manage Jenkins -> Plugins**, verify these are installed and enabled:

- **Git** (`git`)

- **Pipeline** (`workflow-aggregator`)

- **JUnit** (`junit`)

Install missing entries and their dependencies, then restart if requested. `plugins.txt` documents the required plugin IDs; it is not a preinstalled plugin bundle. The exact UI labels can vary slightly across Jenkins releases.

6. Add a separate local build agent

Do not run these builds on the controller's built-in node.

1. Open **Manage Jenkins -> Nodes -> Built-In Node -> Configure**. Set **Number of executors = 0**, and save.

2. Select **New Node**, name it exactly **student-laptop**, and choose **Permanent Agent**.

3. Set **Number of executors = 1**.

4. Set **Remote root directory** to an absolute path dedicated to the lab:

- Windows: `C:\CloudJenkinsLab\jenkins-lab\runtime\agent-work`

- macOS/Linux: your actual `/Users/.../CloudJenkinsLab/jenkins-lab/runtime/agent-work` or `/home/.../CloudJenkinsLab/jenkins-lab/runtime/agent-work`. Do not type `$HOME` literally into the Jenkins UI.

5. Set **Labels = student-laptop**.

6. Set usage to **Only build jobs with label expressions matching this node**.

7. Choose **Launch agent by connecting it to the controller**. Use WebSocket if the UI offers it. Save.

8. Open the node page. Copy **only the agent secret** from its launch command. This secret is a credential: never commit it, screenshot it, or share it.

Terminal C: connect the agent

Windows PowerShell, from `jenkins-lab`:

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\Start-Agent.ps1
```

macOS/Linux:

```
bash start-agent.sh
```

Paste the node secret at the hidden prompt. The launcher downloads `agent.jar` from your local Jenkins, passes the secret via a private temporary file, and removes that file when the process exits normally. If the process is forcibly killed, remove leftover `runtime/agent-private/secret-*` files after confirming the agent has stopped.

Keep Terminal C open. The Jenkins node page should change to **online**. The agent inherits this terminal's PATH: it must be able to run Git and the full JDK.

Security boundary: this lab uses separate processes/workspaces on one trusted laptop. If both run as your same OS account, this is **not strong OS-level isolation**: do not build untrusted repositories or pull requests. Production installations need stronger identity/isolation controls. See: https://www.jenkins.io/doc/book/security/controller-isolation/

7. Create the Pipeline-from-SCM job

1. On the Jenkins dashboard, select **New Item**.
2. Name: **festpass-ci**. Type: **Pipeline**.
3. In the Pipeline section set **Definition = Pipeline script from SCM**.
4. **SCM = Git**.
5. **Repository URL =** `git://127.0.0.1:9418/festpass-java`.
6. **Credentials = none**.
7. **Branch Specifier =** `*/main`.
8. **Script Path =** `Jenkinsfile` (case-sensitive on some systems).
9. Save and select **Build Now once**.

The first run loads the Jenkinsfile and establishes its polling trigger. Expect stages **Checkout -> Test -> Build JAR -> Save artifact**. The console prints the checked-out commit ID and JDK versions. Open the completed build's **Test Result** to see all eight tests. Download `build/festpass.jar` from **Artifacts** and run it locally.

If an optional stage-visualization plugin is absent, the console, test result, build status, and artifacts still demonstrate the pipeline. No extra UI plugin is required for this kit.

8. Make a commit and watch Jenkins react

Edit the success message in `src/main/java/FestPass.java` using the IDE. Save and commit from the **original project folder**, not the Jenkins workspace:

```
git add src/main/java/FestPass.java
git commit -m "Personalize the booking message"
```

The Jenkinsfile polls SCM once each minute. Usually the new build starts within one to two minutes; scheduling or a busy agent can delay it.

No push is needed in this laptop-only arrangement: Jenkins fetches your repository's committed history through the local read-only Git server. File edits without a commit do not trigger it. With a hosted Git remote, commits would also need to be pushed.

Compare `git log -1 --oneline` with the commit printed in Jenkins.

9. Red -> green exercise

1. In `FestPass.canBook`, change < to <=.

2. Commit: `git add src/main/java/FestPass.java`, then `git commit -m "Introduce a deliberate boundary bug"`.

3. Wait for polling. The full-event and zero-capacity tests should fail. Open Test Result / Console Output and explain the failure.

4. Build JAR and Save artifact must be skipped; this failed run must not publish a new JAR. An older green build can still have its own artifact.

5. Restore <, save, and commit with a fix message.

6. Watch the new run turn green and download its new JAR.

This demonstrates **CI and build artifact creation**, not deployment. The app is intentionally not a real multi-user reservation service.

10. Troubleshooting

Symptom	Check / fix
`java` / `javac` not found	Install the full JDK, fix PATH, open fresh terminals. An IDE's bundled Java may not be on PATH.
Unsupported Java / class-file version	Use terminal Java 21 for this pinned Jenkins. Java 17 is too old. Check both controller and agent terminals.
Port 8080 already in use	Choose e.g. 8081 in `Start-Jenkins.ps1 -Port 8081` or `bash start-jenkins.sh 8081`; use the same port in the browser, Jenkins URL, and agent launcher.
Git daemon cannot bind 9418	Stop only your own old lab server, or choose a free port and update the job URL.
`git daemon` is unavailable	Ensure your Git distribution includes it; Linux distributions may package it separately. Ask the instructor/admin if installation is restricted.
Repository not exported / connection refused	Keep Terminal A open, create the initial commit, check extraction paths, and run `git ls-remote` against the printed URL.
Cannot use `git://` under institutional policy	Do not bypass policy. Use an approved remote with the instructor, or Build Now with that approved SCM; hosted remotes need a push.
Waiting for an executor	Agent must be online with label `student-laptop`; terminal C stays open. Keep built-in executors at zero.
Agent unauthorized	Copy the node's **agent secret**, not your administrator password; verify the node name and port.
Git or Java missing only in Jenkins	Restart the agent from a terminal with the correct PATH.
Commit does not trigger a build	Run the job manually once; verify the original repo/branch, committed changes, live Git server, and job Polling Log. Allow scheduling time.
Jenkins tests old code	Compare commit hashes. Do not edit its workspace. Check the repo URL and `main` branch.
No JUnit step / unknown `pipeline` syntax	Install/enable the listed plugins and dependencies; restart as needed.
No test report after compilation error	Fix the compiler error first; compilation failed before the test runner could write XML.
Downloads fail behind a proxy	Use your institution's approved proxy/certificate setup. Do not disable TLS verification.

11. Stop and clean up

Stop **agent terminal C**, then **Jenkins terminal B**, then **Git terminal A** using Ctrl+C. Do not kill unrelated Java processes. No Windows services, scheduled tasks, Docker containers, or public cloud resources were created.

For another session, restart Git, Jenkins, and the agent; the Jenkins home preserves jobs and accounts. The agent secret can be found on its node page. When completely finished, remove only your extracted lab folder after saving anything you want to keep and ensuring its processes have stopped. Never distribute `runtime`, `downloads`, `.git`, passwords, or your configured Jenkins home with a class ZIP.

Official documentation

- Jenkins WAR installation: https://www.jenkins.io/doc/book/installing/war-file/

- Jenkinsfile: https://www.jenkins.io/doc/book/pipeline/jenkinsfile/

- Pipeline syntax and polling: https://www.jenkins.io/doc/book/pipeline/syntax/

- Agents: https://www.jenkins.io/doc/book/using/using-agents/

- JUnit plugin: https://plugins.jenkins.io/junit/

- Git daemon: https://git-scm.com/docs/git-daemon

- Java source-file launch: https://docs.oracle.com/en/java/javase/21/docs/specs/man/java.html